

Main.cpp

```
#include "WeatherApp.h"

int main() {
    WeatherApp app;
    app.run();
    return 0;
}

// g++ -std=c++17 main.cpp WeatherApp.cpp WeatherAnalyzer.cpp Candlestick.cpp CSVReader.cpp
```

WeatherApp.h

```
#pragma once
#include "CSVReader.h"
#include "WeatherAnalyzer.h"
#include "Candlestick.h"
#include <vector>
#include <string>
#include <set>

class WeatherApp {
public:
    // Main run
    void run();

    // Opening the menu
    void runMenu(const std::vector<WeatherRow>& data);

    // Display functions
    void handleDataDisplay(const std::vector<WeatherRow>& data);
    void filterByYear(const std::vector<WeatherRow>& data);
    void filterByMonth(const std::vector<WeatherRow>& data);
    void filterByDay(const std::vector<WeatherRow>& data);
    void displayCandlesticks(const std::vector<Candlestick>& candlesticks, int rangeType) const;

    // Prediction functions
    void handlePrediction(const std::vector<WeatherRow>& data);

    // Country selection function
```

```

    bool selectCountry(const std::vector<WeatherRow>& data);

    // Validation function
    bool isValidCountry(const std::string& country, const
std::vector<WeatherRow>& data) const;

private:
    std::vector<WeatherRow> weatherData;
    std::string selectedCountry;
};

```

WeatherApp.cpp

```

#include "WeatherApp.h"
#include <iostream>
#include <iomanip>
#include <set>
#include <cmath>

// Displays candlestick data
void WeatherApp::displayCandlesticks(const std::vector<Candlestick>&
candlesticks, int rangeType) const {
    std::cout << std::setw(10) << "Date" << std::setw(10) << "Open"
        << std::setw(10) << "High" << std::setw(10) << "Low"
        << std::setw(10) << "Close" << std::endl;

    // Task 1
    for (const auto& candle : candlesticks) {
        std::cout << std::setw(10) << candle.date
            << std::setw(10) << std::fixed << std::setprecision(2) <<
candle.open
            << std::setw(10) <<
static_cast<int>(std::round(candle.high))
            << std::setw(10) << static_cast<int>(std::round(candle.low))
            << std::setw(10) << std::fixed << std::setprecision(2) <<
candle.close
            << std::endl;
    }

    if (candlesticks.empty()) {
        std::cerr << "No candlestick data found for the selected range." <<
std::endl;
    } else {
        // Task 2 with fixed range graph
        std::cout << " " << std::endl;
    }
}

```

```

        std::cout << "Candlestick Plot:" << std::endl;
        CandlestickPlotter::plot(candlesticks, -12.0, 30.0, rangeType);
    }
}

////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
////////////////////////////////////////////////////////////////          Start          //////////////////////////////////////////////////////////////////
////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////

// Choose Country
// Task 3, filter by country
bool WeatherApp::selectCountry(const std::vector<WeatherRow>& data) {
    std::set<std::string> countrySet;
    for (const auto& header : data[0].values) {
        // Get the country code
        countrySet.insert(header.first);
    }

    while (true) { // Loop until valid input is received
        std::cout << "Available Country Code for weather data: ";
        for (const auto& country : countrySet) {
            std::cout << country << " ";
        }
        std::cout << std::endl;

        std::string country;
        std::cout << "Enter the country code (or type 'exit' to quit, 'b' to
go back): ";
        std::cin >> country;

        if (country == "exit") {
            std::cout << "Program exit. Thank you for using!" << std::endl;
            exit(0);
        } else if (country == "b") {
            return false;
        }

        if (countrySet.find(country) != countrySet.end()) {
            selectedCountry = country;
            return true;
        }

        std::cerr << "Invalid country code. Please try again or type 'exit' to
quit, 'b' to go back." << std::endl;
    }
}

```

```

// Yearly Data
void WeatherApp::filterByYear(const std::vector<WeatherRow>& data) {
    auto candlesticks = WeatherAnalyzer::computeCandlestickData(data,
selectedCountry);

    CSVReader::adjustFirstOpenValue(candlesticks, 1);

    // Display historical candlesticks
    displayCandlesticks(candlesticks, 1);
}

// Monthly Data
void WeatherApp::filterByMonth(const std::vector<WeatherRow>& data) {
    std::string input;
    int year;

    while (true) {
        std::cout << "Enter year (1980-2019) or type 'exit' to quit: ";
        std::cin >> input;

        if (input == "exit") {
            std::cout << "Program exit, thank you for using!" << std::endl;
            exit(0);
        }

        try {
            year = std::stoi(input);
            if (year >= 1980 && year <= 2019) {
                break;
            } else {
                std::cerr << "Invalid year. Please enter a year between 1980
and 2019." << std::endl;
            }
        } catch (const std::invalid_argument&) {
            std::cerr << "Invalid input. Please enter a valid year or type
'exit' to quit." << std::endl;
        }
    }

    auto monthlyCandlesticks =
WeatherAnalyzer::computeMonthlyCandlestickData(data, selectedCountry, year);

    if (monthlyCandlesticks.empty()) {
        std::cerr << "No data found for the given year and country." <<
std::endl;
        return;
    }
}

```

```

    CSVReader::adjustFirstOpenValue(monthlyCandlesticks, 2);
    displayCandlesticks(monthlyCandlesticks, 2);
}

// Daily Data
void WeatherApp::filterByDay(const std::vector<WeatherRow>& data) {
    std::string input;
    int year, month;

    while (true) {
        std::cout << "Enter year (1980-2019) or type 'exit' to quit: ";
        std::cin >> input;

        if (input == "exit") {
            std::cout << "Program exit, thank you for using!" << std::endl;
            exit(0);
        }

        try {
            year = std::stoi(input);
            if (year >= 1980 && year <= 2019) {
                break;
            } else {
                std::cerr << "Invalid year. Please enter a year between 1980
and 2019." << std::endl;
            }
        } catch (const std::invalid_argument&) {
            std::cerr << "Invalid input. Please enter a valid year or type
'exit' to quit." << std::endl;
        }
    }

    while (true) {
        std::cout << "Enter month (1-12) or type 'exit' to quit: ";
        std::cin >> input;

        if (input == "exit") {
            std::cout << "Program exit, thank you for using!" << std::endl;
            exit(0);
        }

        try {
            month = std::stoi(input);
            if (month >= 1 && month <= 12) {
                break;
            } else {

```

```

        std::cerr << "Invalid month. Please enter a number between 1
and 12." << std::endl;
    }
    } catch (const std::invalid_argument&) {
        std::cerr << "Invalid input. Please enter a valid month or type
'exit' to quit." << std::endl;
    }
}

    auto dailyCandlesticks =
WeatherAnalyzer::computeDailyCandlestickData(data, selectedCountry, year,
month);

    if (dailyCandlesticks.empty()) {
        std::cerr << "No data found for the given date and country." <<
std::endl;
        return;
    }

    CSVReader::adjustFirstOpenValue(dailyCandlesticks, 3);
    displayCandlesticks(dailyCandlesticks, 3);
}

// Check country in dataset first row
bool WeatherApp::isValidCountry(const std::string& country, const
std::vector<WeatherRow>& data) const {
    std::set<std::string> countrySet;
    if (!data.empty()) {
        for (const auto& header : data[0].values) {
            countrySet.insert(header.first);
        }
    }
    return countrySet.find(country) != countrySet.end();
}

// Main run
void WeatherApp::run() {
    try {
        std::cout << "\nWelcome to TT Weather App!" << std::endl;
        auto data = CSVReader::readWeatherCSV("weather.csv");

        while (true) {
            std::string mode;
            std::cout << "Select mode:\n"
                << "1. Display Data\n"
                << "2. Predict Yearly Data\n"
                << "Enter choice (1 or 2, or 'exit' to quit): ";
            std::cin >> mode;

```

```

        if (mode == "exit") {
            std::cout << "Program exit, thank you for using!" <<
std::endl;
            exit(0);
        } else if (mode == "1") {
            handleDataDisplay(data);
        } else if (mode == "2") {
            // Task 4
            handlePrediction(data);
        } else {
            std::cerr << "Invalid choice. Please enter 1, 2, or 'exit'."
<< std::endl;
        }
    }
} catch (const std::exception& e) {
    std::cerr << "Error: " << e.what() << std::endl;
}
}

// Handles data display
void WeatherApp::handleDataDisplay(const std::vector<WeatherRow>& data) {
    while (true) {
        if (!selectCountry(data)) {
            return;
        }

        // Directly display the menu
        runMenu(data);
    }
}

// Handles prediction
void WeatherApp::handlePrediction(const std::vector<WeatherRow>& data) {
    while (true) {
        if (!selectCountry(data)) {
            return;
        }

        auto candlesticks = WeatherAnalyzer::computeCandlestickData(data,
selectedCountry);

        if (candlesticks.empty()) {
            std::cerr << "No data found for the selected country." <<
std::endl;
        } else {
            double prediction =
WeatherAnalyzer::predictYearlyWeather(candlesticks);

```

```

        std::cout << "\nPrediction for upcoming year:" << std::endl;
        // Above 0 is hot other value is cold
        std::cout << "Overall Weather: " << (prediction > 0 ? "Hot" :
"Cold") << std::endl;
        std::cout << "Predicted Close Value: " << std::fixed <<
std::setprecision(2) << prediction << std::endl;
    }

    std::string input;
    std::cout << "Type anything to continue or else enter ('b' to go back
to main menu or 'exit' to quit): ";
    std::cin >> input;

    if (input == "b") {
        return;
    }

    if (input == "exit") {
        std::cout << "Program exit, thank you for using!" << std::endl;
        exit(0);
    }
}
}

// Runs the menu
void WeatherApp::runMenu(const std::vector<WeatherRow>& data) {
    while (true) {
        std::string input;
        // Task 3, filter by date range
        std::cout << std::endl << "Select date range type:" << std::endl
        << "1. Yearly" << std::endl
        << "2. Monthly" << std::endl
        << "3. Daily" << std::endl
        << "4. Back to Country Selection" << std::endl;

```

Table of Contents

Main.cpp	1
WeatherApp.h.....	1
WeatherApp.cpp.....	2
WeatherAnalyzer.h	10
WeatherAnalyzer.cpp.....	10

```

    while (true) {
        std::cout << "Enter choice (from 1 to 4 or type 'exit' to quit):
";

```

```

        std::cin >> input;

        if (input == "exit") {
            std::cout << "Program exit, thank you for using!" <<
std::endl;
            exit(0); // Terminate the program
        }

        int choice;
        try {
            choice = std::stoi(input);
            if (choice >= 1 && choice <= 4) {
                break;
            } else {
                std::cerr << "Invalid choice. " << std::endl;
            }
        } catch (const std::invalid_argument&) {
            std::cerr << "Invalid input. " << std::endl;
        }
    }

    int choice = std::stoi(input);
    switch (choice) {
        case 1:
            filterByYear(data);
            break;
        case 2:
            filterByMonth(data);
            break;
        case 3:
            filterByDay(data);
            break;
        case 4:
            std::cout << " " << std::endl;
            std::cout << "Back to country selection" << std::endl;
            return;
    }
}
}

```

```

////////////////////////////////////
////////////////////////////////////      END      //////////////////////////////////
////////////////////////////////////

```

WeatherAnalyzer.h

```
#pragma once
#include "Candlestick.h"
#include "CSVReader.h"
#include <vector>
#include <string>

class WeatherAnalyzer {
public:
    static std::vector<Candlestick> computeCandlestickData(const
std::vector<WeatherRow>& data, const std::string& country);
    static std::vector<Candlestick> computeMonthlyCandlestickData(const
std::vector<WeatherRow>& data, const std::string& country, int year);
    static std::vector<Candlestick> computeDailyCandlestickData(const
std::vector<WeatherRow>& data, const std::string& country, int year, int
month);
    static double predictYearlyWeather(const std::vector<Candlestick>&
candlesticks);

private:
    // Helper function to calculate high, low, and mean values
    static void computeStatistics(const std::vector<double>& values, double&
high, double& low, double& mean);
};
```

WeatherAnalyzer.cpp

```
#include "WeatherAnalyzer.h"
#include <limits>
#include <numeric>
#include <algorithm>
#include <iostream>
#include <iomanip>

////////////////////////////////////
// Start
////////////////////////////////////

// Computation Function
void WeatherAnalyzer::computeStatistics(const std::vector<double>& values,
double& high, double& low, double& mean) {
```

```

        if (!values.empty()) {
            high = *std::max_element(values.begin(), values.end());
            low = *std::min_element(values.begin(), values.end());
            mean = std::accumulate(values.begin(), values.end(), 0.0) /
values.size();
        } else {
            high = low = mean = 0.0;
        }
    }
}

// Computes yearly candlestick data
std::vector<Candlestick> WeatherAnalyzer::computeCandlestickData(
    const std::vector<WeatherRow>& data, const std::string& country) {
    std::vector<Candlestick> candlesticks;
    std::map<int, std::vector<double>> yearlyData;

    // Group temperatures by year
    for (const auto& entry : data) {
        try {
            int year = std::stoi(entry.timestamp.substr(0, 4));
            if (entry.values.find(country) != entry.values.end()) {
                yearlyData[year].push_back(entry.values.at(country));
            }
        } catch (const std::exception& e) {
            std::cerr << "Error processing with timestamp: " <<
entry.timestamp << std::endl;
        }
    }

    double previousClose = 0.0;
    for (int year = 1980; year <= 2019; ++year) {
        double open = previousClose, high = 0.0, low = 0.0, close = 0.0;

        if (yearlyData.count(year)) {
            computeStatistics(yearlyData[year], high, low, close);
        }

        candlesticks.emplace_back(std::to_string(year), open, high, low,
close);
        previousClose = close;
    }

    return candlesticks;
}

////////////////////////////////////
////////////////////////////////////      END      //////////////////////////////////
////////////////////////////////////

```

```

// Computes monthly candlestick data
std::vector<Candlestick> WeatherAnalyzer::computeMonthlyCandlestickData(
    const std::vector<WeatherRow>& data,
    const std::string& country,
    int year) {
    std::vector<Candlestick> candlesticks;
    std::map<int, std::vector<double>> monthlyData;

    // Group data by month
    for (const auto& entry : data) {
        int entryYear = std::stoi(entry.timestamp.substr(0, 4));
        int entryMonth = std::stoi(entry.timestamp.substr(5, 2));
        if (entryYear == year && entry.values.find(country) !=
entry.values.end()) {
            monthlyData[entryMonth].push_back(entry.values.at(country));
        }
    }

    double previousClose = 0.0;

    for (int month = 1; month <= 12; ++month) {
        double open = 0.0;
        double close = 0.0;
        double high = std::numeric_limits<double>::lowest();
        double low = std::numeric_limits<double>::max();

        if (monthlyData.find(month) != monthlyData.end()) {
            const auto& temperatures = monthlyData[month];

            // Compute close (current month's mean)
            close = std::accumulate(temperatures.begin(), temperatures.end(),
0.0) / temperatures.size();

            // Compute high and low
            high = *std::max_element(temperatures.begin(),
temperatures.end());
            low = *std::min_element(temperatures.begin(), temperatures.end());
        }

        // Set the open value
        if (month == 1) {
            if (year == 1980) {
                // Since there is no data for 1979, 1980 Jan open value is set
to close value + 0.4
                open = close + 0.4;
            } else {
                // For other years, use December mean of the previous year

```

```

        std::string prevDecember = std::to_string(year - 1) + "-12";
        std::vector<double> prevYearTemps;
        for (const auto& entry : data) {
            if (entry.timestamp.substr(0, 7) == prevDecember &&
                entry.values.find(country) != entry.values.end()) {
                prevYearTemps.push_back(entry.values.at(country));
            }
        }
        if (!prevYearTemps.empty()) {
            open = std::accumulate(prevYearTemps.begin(),
prevYearTemps.end(), 0.0) / prevYearTemps.size();
        } else {
            open = close + 0.4;
        }
    }
} else {
    // For other months, use the previous month's close
    open = previousClose;
}

// Add the candlestick for the month
candlesticks.emplace_back(
    std::to_string(year) + "-" + (month < 10 ? "0" : "") +
std::to_string(month),
    open, high, low, close);

// Update previousClose for the next month
previousClose = close;
}

return candlesticks;
}

// Computes daily candlestick data
std::vector<Candlestick> WeatherAnalyzer::computeDailyCandlestickData(
    const std::vector<WeatherRow>& data, const std::string& country, int year,
int month) {
    std::vector<Candlestick> dailyData;
    std::map<int, std::vector<double>> dailyTemperatures;

    // Some months dont' have 31 days
    auto daysInMonth = [](int year, int month) {
        // Calculate leap year
        if (month == 2) {
            return (year % 4 == 0 && year % 100 != 0) || (year % 400 == 0) ?
29 : 28;
        }
    }
}

```

```

        return (month == 4 || month == 6 || month == 9 || month == 11) ? 30 :
31;
    };

    int maxDays = daysInMonth(year, month);
    for (const auto& row : data) {
        try {
            int rowYear = std::stoi(row.timestamp.substr(0, 4));
            int rowMonth = std::stoi(row.timestamp.substr(5, 2));
            int rowDay = std::stoi(row.timestamp.substr(8, 2));

            if (rowYear == year && rowMonth == month &&
row.values.find(country) != row.values.end()) {
                dailyTemperatures[rowDay].push_back(row.values.at(country));
            }
        } catch (const std::exception& e) {
            std::cerr << "Error processing with timestamp: " << row.timestamp
<< std::endl;
        }
    }

    double previousClose = 0.0;
    for (int day = 1; day <= maxDays; ++day) {
        double open = previousClose, high = 0.0, low = 0.0, close = 0.0;

        if (dailyTemperatures.count(day)) {
            computeStatistics(dailyTemperatures[day], high, low, close);
        }

        dailyData.emplace_back(
            std::to_string(year) + "-" + (month < 10 ? "0" : "") +
std::to_string(month) + "-" +
            (day < 10 ? "0" : "") + std::to_string(day),
            open, high, low, close);
        previousClose = close;
    }

    return dailyData;
}

////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
////////////////////////////////////////////////////////////////                      Start                      //
////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////

double WeatherAnalyzer::predictYearlyWeather(const std::vector<Candlestick>&
candlesticks) {
    if (candlesticks.empty()) {

```

```

        throw std::runtime_error("No data available for prediction.");
        // Alternatively, you could return a default value, e.g.:
        // return 0.0;
    }

    std::vector<double> weights = {2, 5, 8, 11, 14, 17, 20, 23}; // Adjust
weight percentages
    size_t weightCount = weights.size();
    double weightedSum = 0.0, weightTotal = 0.0;

    for (size_t i = 0; i < candlesticks.size() && i < weightCount; ++i) {
        double weight = weights[weightCount - i - 1]; // Most recent year gets
highest weight
        weightedSum += candlesticks[candlesticks.size() - i - 1].close *
weight;
        weightTotal += weight;
    }

    return weightedSum / weightTotal; // Return predicted close value
}

////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
////////////////////////////////////////////////////////////////                      END                      //
////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////

```

CSVReader.h

```

#pragma once
#include <vector>
#include <string>
#include <map>
#include "Candlestick.h"

// Define the WeatherRow structure
struct WeatherRow {
    std::string timestamp;
    std::map<std::string, double> values;
};

class CSVReader {
public:
    CSVReader();
    static std::vector<WeatherRow> readWeatherCSV(const std::string&
csvFilename);

```

```

        static std::vector<std::string> tokenise(const std::string& csvLine, char
separator);
        static void adjustFirstOpenValue(std::vector<Candlestick>& candlesticks,
int rangeType);
};

```

CSVReader.cpp

```

#include "CSVReader.h"
#include <iostream>
#include <fstream>
#include <stdexcept>

////////////////////////////////////
//////////////////////////////////// Start //////////////////////////////////////
////////////////////////////////////

CSVReader::CSVReader() {}

std::vector<WeatherRow> CSVReader::readWeatherCSV(const std::string&
csvFilename) {
    std::vector<WeatherRow> data;
    std::ifstream csvFile(csvFilename);
    std::string line;

    if (!csvFile.is_open()) {
        throw std::runtime_error("Could not open file: " + csvFilename);
    }

    // Get Country Code
    std::vector<std::string> headers;
    if (std::getline(csvFile, line)) {
        headers = tokenise(line, ',');
        for (auto& header : headers) {
            if (header.size() > 12 && header.substr(header.size() - 12) ==
"_temperature") {
                header = header.substr(0, header.size() - 12);
            }
        }
    }

    while (std::getline(csvFile, line)) {
        auto tokens = tokenise(line, ',');
        if (tokens.size() != headers.size()) {

```

```

        std::cerr << "CSVReader::readWeatherCSV - Missing columns in line:
" << line << std::endl;
        continue;
    }

    WeatherRow row;
    row.timestamp = tokens[0];
    try {
        for (size_t i = 1; i < tokens.size(); ++i) {
            row.values[headers[i]] = std::stod(tokens[i]);
        }
        data.push_back(row);
    } catch (const std::exception& e) {
        std::cerr << "CSVReader::readWeatherCSV - Failed to convert in
line: " << line << std::endl;
    }
}

return data;
}

void CSVReader::adjustFirstOpenValue(std::vector<Candlestick>& candlesticks,
int rangeType) {
    const std::string& firstDate = candlesticks[0].date; // Extract once for
reuse

    switch (rangeType) {
        case 1: // Yearly
            if (firstDate.substr(0, 4) == "1980") {
                candlesticks[0].open = candlesticks[0].close + 0.4;
            }
            break;

        case 2: // Monthly
            if (firstDate.substr(0, 7) == "1980-01") {
                candlesticks[0].open = candlesticks[0].close + 0.4;
            }
            break;

        case 3: // Daily
            // Every first day
            if (firstDate.size() >= 10 && firstDate.substr(8, 2) == "01") {
                candlesticks[0].open = candlesticks[0].close + 0.4;
            }
            break;
    }
}
}

```

```

std::vector<std::string> CSVReader::tokenise(const std::string& csvLine, char
separator) {
    std::vector<std::string> tokens;
    size_t start = 0, end = 0;

    while ((end = csvLine.find(separator, start)) != std::string::npos) {
        tokens.push_back(csvLine.substr(start, end - start));
        start = end + 1;
    }

    tokens.push_back(csvLine.substr(start));
    return tokens;
}

```

```

////////////////////////////////////
////////////////////////////////////      END      //////////////////////////////////
////////////////////////////////////

```

Candlestick.h

```

#pragma once
#include <vector>
#include <string>

class Candlestick {
public:
    std::string date;
    double open;
    double high;
    double low;
    double close;

    Candlestick(std::string date, double open, double high, double low, double
close);
};

class CandlestickPlotter {
public:
    static void plot(const std::vector<Candlestick>& candlesticks, double
minRange, double maxRange, int rangeType);
};

```

Candlestick.cpp

```
#include "Candlestick.h"
#include <iostream>
#include <iomanip>
#include <cmath>
#include <algorithm>

// Constructor for Candlestick
Candlestick::Candlestick(std::string date, double open, double high, double low, double close)
    : date(date), open(open), high(high), low(low), close(close) {}

// Function to plot candlesticks
void CandlestickPlotter::plot(const std::vector<Candlestick>& candlesticks,
double minRange, double maxRange, int rangeType) {
    if (candlesticks.empty()) {
        std::cerr << "No data available for plotting." << std::endl;
        return;
    }

    const int plotHeight = 30; // Number of rows for the vertical plot

    // Draw Graph based on these values
    double maxHigh = std::numeric_limits<double>::lowest();
    double minLow = std::numeric_limits<double>::max();
    double maxOpen = std::numeric_limits<double>::lowest();
    double minOpen = std::numeric_limits<double>::max();

    for (const auto& candle : candlesticks) {
        maxHigh = std::max(maxHigh, candle.high);
        minLow = std::min(minLow, candle.low);
        maxOpen = std::max(maxOpen, candle.open);
        minOpen = std::min(minOpen, candle.open);
    }

    // Add padding to high and low values for more accuracy
    double graphMax = maxHigh + 5;
    double graphMin = minLow - 5;

    // Generate axis labels
    std::vector<double> axisLabels;

    // Top values range
    for (double val = graphMax; val >= maxOpen; val -= 1.5) {
        axisLabels.push_back(val);
    }
}
```

```

// Top values range
for (double val = maxOpen; val > minOpen; val -= 0.2) {
    axisLabels.push_back(val);
}

// Bottom values range
for (double val = minOpen - 0.1; val >= graphMin; val -= 1.5) {
    axisLabels.push_back(val);
}

// Generate the plot grid
std::vector<std::string> plot(axisLabels.size(),
std::string(candlesticks.size() * 6, ' '));

for (size_t i = 0; i < candlesticks.size(); ++i) {
    const auto& candle = candlesticks[i];

    auto findPosition = [&](double value) {
        auto it = std::lower_bound(axisLabels.begin(), axisLabels.end(),
value, std::greater<>());
        return std::distance(axisLabels.begin(), it);
    };

    int openPos = std::clamp(static_cast<int>(findPosition(candle.open)),
0, static_cast<int>(axisLabels.size() - 1));

    int closePos =
std::clamp(static_cast<int>(findPosition(candle.close)),
0, static_cast<int>(axisLabels.size() - 1));

    int highPos = std::clamp(static_cast<int>(findPosition(candle.high)),
0, static_cast<int>(axisLabels.size() - 1));

    int lowPos = std::clamp(static_cast<int>(findPosition(candle.low)),
0, static_cast<int>(axisLabels.size() - 1));

    // Draw high and low (stalk)
    for (int j = highPos; j <= lowPos; ++j) {
        plot[j][i * 6 + 3] = '|';
    }

    // Draw open marks
    plot[openPos][i * 6 + 2] = '(';
    plot[openPos][i * 6 + 3] = 'o';
    plot[openPos][i * 6 + 4] = ')';

    // Draw close marks
    plot[closePos][i * 6 + 2] = '(';
    plot[closePos][i * 6 + 3] = 'c';
    plot[closePos][i * 6 + 4] = ')';
}

```

```

    }

    // Print the plot with vertical temperature axis
    for (size_t j = 0; j < axisLabels.size(); ++j) {
        double value = axisLabels[j];
        std::cout << std::setw(6) << std::fixed << std::setprecision(1) <<
value << " |";

        for (size_t k = 0; k < plot[j].size(); ++k) {
            std::cout << plot[j][k];
        }
        std::cout << std::endl;
    }

    // Print the horizontal axis
    std::cout << std::string(candlesticks.size() * 7, '_') << std::endl;
    std::cout << "          ";
    for (size_t i = 0; i < candlesticks.size(); ++i) {
        // Yearly
        if (rangeType == 1) {
            // more entries than other 2
            std::cout << std::setw(6) << candlesticks[i].date.substr(0, 4);
        }
        // Monthly and daily
        else if (rangeType == 2 || rangeType == 3) {
            std::cout << std::setw(6) << (i + 1);
        }
    }
    std::cout << std::endl;
}

```